

MATLAB Plotting Script: Nyquist Diagram

Chat transcript, 2 October 2026

You

Take this existing code and make a third plot that is in the same window. The third plot will be a Nyquist diagram. The Nyquist diagram needs an input signal frequency and sample frequency, both in Hz. Use these frequencies to calculate the plot. Only put as many lines in the Nyquist diagram as necessary plus a few more for good measure. Make the Nyquist diagram set up to look like the attached photo of a sample Nyquist diagram.

```
% Plots the DAT (time vs. voltage) and FFT (frequency vs. voltage) files
% for one experiment, both in the same figure window.
% Files: 9_17_2026jamesXY.txt, X = experiment number, Y = 'dat' or 'fft'.
% Each file: tab-delimited, 1 header line, 2 columns.
clear; clc; close all;

%% ===== USER SETTINGS =====
expNum      = 11;                % experiment number (integer)
dataFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Matlab'; % folder containing
the .txt files

% ---- OPTION B: specify file names manually instead ----
useManualNames = false;
FFT = "fft.txt";
DAT = "dat.txt";

% ---- DAT plot axis limits (override) ----
% Leave as [] to autoscale to the min/max of the data.
% Otherwise use [min max], e.g. timeLim = [0 0.005]; voltLim = [-3 3];
timeLim = [0 0.01];             % X axis (time, s)
voltLim = [-5 5];               % Y axis (voltage, V)
%% =====

%% Build file paths
if useManualNames
    datFile = fullfile(dataFolder, DAT);
    fftFile = fullfile(dataFolder, FFT);
else
    prefix = sprintf('9_17_2026james%d', expNum);
    datFile = fullfile(dataFolder, [prefix 'dat.txt']);
    fftFile = fullfile(dataFolder, [prefix 'fft.txt']);
end

if ~isfile(datFile), error('DAT file not found: %s', datFile); end
if ~isfile(fftFile), error('FFT file not found: %s', fftFile); end

%% Read data (1 header line, tab-delimited)
datData = readmatrix(datFile, 'NumHeaderLines', 1, 'Delimiter', '\t');
fftData = readmatrix(fftFile, 'NumHeaderLines', 1, 'Delimiter', '\t');

t = datData(:,1); % time [s]
vt = datData(:,2); % voltage [V]
f = fftData(:,1); % frequency [Hz]
vf = fftData(:,2); % voltage [V]

%% Plot both in one figure
figure('Name', sprintf('Experiment %d', expNum), 'NumberTitle', 'off');
tl = tiledlayout(2, 1, 'TileSpacing', 'compact');
title(tl, sprintf('Experiment %d', expNum));

% --- Plot 1: Time vs. Voltage (DAT) ---
nexttile;
plot(t, vt, 'LineWidth', 1.2);
xlabel('Time (s)');
```

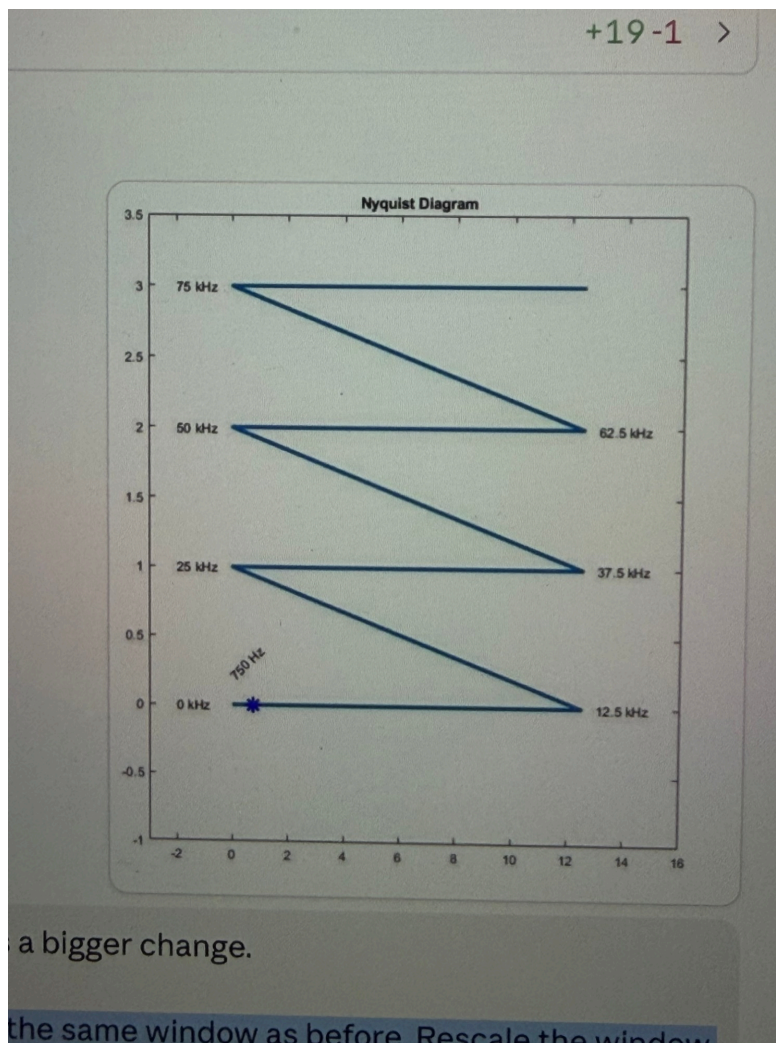
```

ylabel('Voltage (V)');
title('Time vs. Voltage');
grid on;
xlim(getLimits(t, timeLim));
ylim(getLimits(vt, voltLim));

% --- Plot 2: Frequency vs. Voltage (FFT) ---
nexttile;
plot(f, vf, 'LineWidth', 1.2);
xlabel('Frequency (Hz)');
ylabel('Voltage (V)');
title('Frequency vs. Voltage');
grid on;
axis tight;

%% Local function: autoscale to data min/max unless user override is given
function lim = getLimits(x, userLim)
    if isempty(userLim)
        lim = [min(x) max(x)];
        if lim(1) == lim(2)
            lim = lim + [-1 1];
        end
    else
        lim = userLim;
    end
end

```



Claude

This was already done in an earlier session. The repo has `plotExperiment.m` with the Nyquist diagram added, so I didn't change anything this time. I did push the branch `claude/practical-franklin-otp2mz` so the file is on GitHub. There's no MATLAB here, so I haven't run it. Run it once to check it.

What it does:

- **Inputs:** `signal = 750` (signal frequency, Hz) and `source = 25000` (sample frequency, Hz). With these defaults the plot should match your example picture.
- **Layout:** time vs. voltage sits above frequency vs. voltage on the left. The Nyquist diagram is on the right and is as tall as both.
- **Shape:** one zig-zag line. Each flat line runs from 0 to half the sample frequency (12.5 kHz in your picture). A diagonal then runs back up to the left end of the next line. The frequency is labelled at both ends of each flat line (0 / 12.5 kHz, 25 / 37.5 kHz, ...). The labels and x-axis switch between Hz, kHz and MHz to suit the sample frequency.
- **Number of lines:** it draws the lines needed to reach the signal frequency, plus 3 more. That gives the 4 lines in your picture. Change `nExtra` to draw more or fewer.
- **Signal:** a blue * with the frequency written at 45°, like the "750 Hz" in your picture.
- **One addition:** your picture doesn't show this. If the signal is above half the sample frequency, a red dashed line drops from the star to the bottom line. A red dot there is labelled with the aliased frequency.
- **Show or hide plots:** `showTimeDomain`, `showFreqDomain` and `showNyquist` turn each plot on or off. A data file is only read if its plot is on.

Paste it into a single `.m` file. You need MATLAB R2019b or newer for `tiledlayout`. If you get an error that `getLimits` has already been declared, there are two copies of that function in the file. Replace the whole file with this:

```
% Plots the DAT (time vs. voltage) and FFT (frequency vs. voltage) files
% for one experiment, plus an optional Nyquist (folding) diagram, all in
% the same figure window.
% Files: 9_17_2026jamesXY.txt, X = experiment number, Y = 'dat' or 'fft'.
% Each file: tab-delimited, 1 header line, 2 columns.
clear; clc; close all;

%% ===== USER SETTINGS =====
expNum      = 11;                % experiment number (integer)
dataFolder  = 'C:\Users\jsasso\OneDrive - Syracuse University\MAE Matlab'; % folder containing
the .txt files

% ---- OPTION B: specify file names manually instead ----
useManualNames = false;
FFT = "fft.txt";
DAT = "dat.txt";

% ---- Which plots to show (true/false) ----
showTimeDomain = true;          % Time vs. Voltage (DAT file)
showFreqDomain = true;          % Frequency vs. Voltage (FFT file)
showNyquist     = true;          % Nyquist (folding) diagram

% ---- Nyquist diagram inputs (both in Hz) ----
signal = 750;                   % signal frequency [Hz]
source = 25000;                  % sample frequency [Hz]

% ---- DAT plot axis limits (override) ----
% Leave as [] to autoscale to the min/max of the data.
% Otherwise use [min max], e.g. timeLim = [0 0.005]; voltLim = [-3 3];
timeLim = [0 0.01];              % X axis (time, s)
voltLim = [-5 5];                % Y axis (voltage, V)
```

```

%% =====

nPlots = showTimeDomain + showFreqDomain + showNyquist;
if nPlots == 0, error('No plots selected. Set at least one of the show* flags to true.');
```

end

```

%% Build file paths
if useManualNames
    datFile = fullfile(dataFolder, DAT);
    fftFile = fullfile(dataFolder, FFT);
else
    prefix = sprintf('9_17_2026james%d', expNum);
    datFile = fullfile(dataFolder, [prefix 'dat.txt']);
    fftFile = fullfile(dataFolder, [prefix 'fft.txt']);
end

%% Read data (1 header line, tab-delimited) -- only the files that are needed
if showTimeDomain
    if ~isfile(datFile), error('DAT file not found: %s', datFile); end
    datData = readmatrix(datFile, 'NumHeaderLines', 1, 'Delimiter', '\t');
    t = datData(:,1); % time [s]
    vt = datData(:,2); % voltage [V]
end
if showFreqDomain
    if ~isfile(fftFile), error('FFT file not found: %s', fftFile); end
    fftData = readmatrix(fftFile, 'NumHeaderLines', 1, 'Delimiter', '\t');
    f = fftData(:,1); % frequency [Hz]
    vf = fftData(:,2); % voltage [V]
end

%% Figure and layout
% 1 plot -> single tile
% 2 plots -> stacked vertically (order: time, frequency, nyquist)
% 3 plots -> time over frequency on the left, nyquist on the right spanning both rows
fig = figure('Name', sprintf('Experiment %d', expNum), 'NumberTitle', 'off');
if nPlots == 3
    fig.Position = [100 100 1400 700];
    tl = tiledlayout(2, 2, 'TileSpacing', 'compact');
    tiles = {1, 3, 2}; % tile index for time, freq, nyquist
    nyqSpan = [2 1];
else
    fig.Position = [100 100 800 350*nPlots];
    tl = tiledlayout(nPlots, 1, 'TileSpacing', 'compact');
    tiles = num2cell(cumsum([showTimeDomain showFreqDomain showNyquist]));
    nyqSpan = [1 1];
end
title(tl, sprintf('Experiment %d', expNum));

% --- Plot 1: Time vs. Voltage (DAT) ---
if showTimeDomain
    nexttile(tiles{1});
    plot(t, vt, 'LineWidth', 1.2);
    xlabel('Time (s)');
    ylabel('Voltage (V)');
    title('Time vs. Voltage');
    grid on;
    xlim(getLimits(t, timeLim));
    ylim(getLimits(vt, voltLim));
end

% --- Plot 2: Frequency vs. Voltage (FFT) ---
if showFreqDomain
    nexttile(tiles{2});
    plot(f, vf, 'LineWidth', 1.2);
    xlabel('Frequency (Hz)');
    ylabel('Voltage (V)');
    title('Frequency vs. Voltage');
    grid on;
    axis tight;
end

% --- Plot 3: Nyquist (folding) diagram ---

```

```

if showNyquist
    ax = nexttile(tiles{3}, nyqSpan);
    plotNyquist(ax, signal, source);
end

%% Local function: autoscale to data min/max unless user override is given
function lim = getLimits(x, userLim)
    if isempty(userLim)
        lim = [min(x) max(x)];
        if lim(1) == lim(2)
            lim = lim + [-1 1];
        end
    else
        lim = userLim;
    end
end

%% Local function: Nyquist folding diagram
% Zig-zag of the frequency axis folded at multiples of fN = fs/2:
% level m (y = m) is a horizontal rung from 2m*fN (left) to (2m+1)*fN (right),
% and a diagonal runs from (2m+1)*fN back up-left to 2(m+1)*fN on the next level.
% A frequency on any segment aliases to the frequency directly below it on
% the bottom rung (0 .. fN).
function plotNyquist(ax, fSig, fs)
    if fs <= 0, error('Sample frequency (source) must be > 0 Hz.');
```

```

    end
    if fSig < 0, error('Signal frequency (signal) must be >= 0 Hz.');
```

```

    end
    fN = fs/2;
    [uScale, uName] = pickUnits(fs); % display units for labels / x axis
```

```

    % Which band (k) the signal is in, and where along it (r = 0..1)
    k = floor(fSig/fN);
    r = fSig/fN - k;
    if k > 0 && r == 0 % exactly on a fold: use end of previous band
        k = k - 1;
        r = 1;
    end
```

```

    m = floor(k/2); % level the signal's band starts on
    if mod(k, 2) == 0 % horizontal rung, left -> right
        xSig = r*fN;
        ySig = m;
    else % diagonal, right -> up-left
        xSig = (1 - r)*fN;
        ySig = m + r;
    end
```

```

    fAlias = xSig;
```

```

    nExtra = 3; % extra levels drawn above the signal for clarity
    nLevels = m + 1 + nExtra;
```

```

    % Zig-zag path: (0,0) -> (fN,0) -> (0,1) -> (fN,1) -> ...
    xp = repmat([0 fN], 1, nLevels);
    yp = repelem(0:nLevels-1, 2);
```

```

    hold(ax, 'on');
    plot(ax, xp/uScale, yp, '-', 'LineWidth', 2, 'Color', [0 0.3 0.6]);
```

```

    % Level labels: left end = 2m*fN, right end = (2m+1)*fN
    dx = 0.08*fN/uScale;
    for lv = 0:nLevels-1
        text(ax, -dx, lv, fmtFreq(2*lv*fN, uScale, uName), ...
            'HorizontalAlignment', 'right', 'VerticalAlignment', 'middle');
        text(ax, fN/uScale + dx, lv, fmtFreq((2*lv+1)*fN, uScale, uName), ...
            'HorizontalAlignment', 'left', 'VerticalAlignment', 'middle');
    end
```

```

    % Signal frequency marker
    plot(ax, xSig/uScale, ySig, '*', 'Color', 'b', 'MarkerSize', 12, 'LineWidth', 1.5);
    text(ax, xSig/uScale, ySig + 0.08, [' ' fmtFreq(fSig, 1, 'Hz')], ...
        'Rotation', 45, 'Color', 'b', 'HorizontalAlignment', 'left', ...
        'VerticalAlignment', 'bottom');
```

```

% If the signal is above fN, project it down to the bottom rung (alias)
if fSig > fN
    plot(ax, [xSig xSig]/uScale, [ySig 0], 'r--', 'LineWidth', 1.2);
    plot(ax, fAlias/uScale, 0, 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7);
    text(ax, fAlias/uScale, -0.08, [' ' fmtFreq(fAlias, 1, 'Hz') ' (alias)'], ...
        'Rotation', -45, 'Color', 'r', 'HorizontalAlignment', 'left', ...
        'VerticalAlignment', 'top');
end
hold(ax, 'off');

xlim(ax, [-0.25*fN 1.3*fN]/uScale);
ylim(ax, [-1, nLevels - 0.5]);
box(ax, 'on');
title(ax, 'Nyquist Diagram');
end

function [scale, name] = pickUnits(f)
    if f >= 1e6
        scale = 1e6; name = 'MHz';
    elseif f >= 1e3
        scale = 1e3; name = 'kHz';
    else
        scale = 1; name = 'Hz';
    end
end

function s = fmtFreq(f, scale, name)
    s = sprintf('%g %s', f/scale, name);
end

```

You

make this chat a pdf